

Unique Decimal Count

You are given a binary string **binary** consisting only of '0' and '1'. Consider all non-empty subsequences of the string.

- A subsequence is formed by deleting zero or more characters without changing the order of the remaining characters.
- The binary string may contain leading zeros.
- Each subsequence is interpreted as a binary number and converted to its decimal value.

Determine the number of **distinct decimal values** that can be formed from all such non-empty subsequences.

Return the count of unique decimal values.

Constraints

- $1 \leq \text{length of } \textit{binary} \leq 20$
- The string *binary* consists of characters '0' and '1' only.

Sample Cases

Sample Case 0: `binary = "011"`

Output: 3

Explanation: Distinct subsequences are "0", "1", "01", and "11". Their decimal values are 0, 1, 1, and 3. The distinct decimal numbers are {0, 1, 3}.

Sample Case 1: `binary = "11"`

Output: 2

Explanation: Distinct subsequences are "1" and "11". Their decimal values are 1 and 3. The distinct decimal numbers are {1, 3}.

Java Implementation

```
import java.io.*;
import java.util.*;

class Result {

    /*
     * Complete the 'getCount' function below.
     * The function is expected to return an INTEGER.
     * The function accepts STRING binary as parameter.
     */

    public static int getCount(String binary) {
        int n = binary.length();
        Set<Integer> uniqueValue = new HashSet<>();

        // Iterate through all possible bitmask combinations
        for (int mask = 1; mask < (1 << n); mask++) {
            int value = 0;
            for (int i = 0; i < n; i++) {
                // If the i-th bit of mask is set, include binary[i] in subsequence
                if ((mask & (1 << i)) != 0) {
                    value = (value << 1) + (binary.charAt(i) - '0');
                }
            }
            uniqueValue.add(value);
        }

        return uniqueValue.size();
    }
}
```